

SmartHire ML Service

Process Description, Modular Architecture & API Specification

Version: 1.0.0	Runtime: Python 3.11+ / FastAPI	Framework: Uvicorn / Pytest
Service Boundary: Microservice (REST)	ML & AI: scikit-learn, PyMuPDF, Gemini	Verification: 94 Automated Tests Passed

1. System Architectural Overview

SmartHire ML Service is a dedicated, stateless Python microservice designed for AI-assisted candidate screening, resume document parsing, controlled technical skill extraction, TF-IDF vectorization, multi-criteria candidate scoring, and deterministic ranking. It interacts with the primary Node.js/TypeScript backend exclusively over HTTP/REST.

Core Design Principles: • AI-Assisted, Not Autonomous: Provides scoring insights and transparent explanations to human recruiters; never autonomously rejects candidates. • Fairness & Privacy: Protected characteristics (gender, race, age, religion) are never extracted, stored, or utilized in ranking. • No Database Dependency: The service owns no direct PostgreSQL connection; all candidate/job states are passed via REST payloads. • Zero Stack Trace Leakage: Domain exceptions are mapped to unified JSON error models.

2. End-to-End Processing Pipeline

When candidate resumes and job descriptions are screened, the data flows sequentially through 6 core phases:

Phase	Module / Service	Description & Algorithmic Process
1. Ingestion & Validation	app.services.document_service	Accepts PDF or DOCX file streams (<=10MB). Inspects binary magic bytes (%PDF or PK ZIP)
2. NLP Preprocessing	app.nlp.normalizer app.nlp.tokenizer app.nlp.section_parser	Cleans Unicode, strips invalid control chars, parses canonical sections (Summary, Experience)
3. Skill Extraction	app.services.skill_service app.data.skills app.data.mappings	Matches candidate text against controlled dictionary of 70+ canonical technical skills. Automate
4. Feature Vectorization	app.ml.tfidf app.services.vectorization_service	Converts preprocessed resumes and job requirements into numerical vectors using sublinear
5. Multi-Criteria Scoring	app.ml.scoring app.services.scoring_service	Computes four weighted component scores: Skill Match (50%), Experience Match (25%), Edu
6. Ranking & Explanation	app.ml.ranking app.services.explanation_service	Applies deterministic tie-breaking to batch-ordered candidates. Produces human-interpretable

3. Scoring Mathematical Formulation & Thresholds

Candidate evaluation uses a transparent, explainable formula rather than an opaque black box. The composite score (0 to 100) is calculated as:

Overall Score Formula:
Score = (0.50 × Skill_Score) + (0.25 × Experience_Score) + (0.15 × Education_Score) + (0.10 × Semantic_Similarity)

- **Skill Score:** $|Candidate_Skills \cap Job_Skills| / |Job_Skills| \times 100$
- **Experience Score:** $\min(1.0, Candidate_Years / Required_Years) \times 100$
- **Education Score:** Compatibility level matching (Exact: 100%, Exceeds: 100%, Related/Lower: 70%, None: 40%)
- **Semantic Similarity:** Cosine similarity of TF-IDF vectors (Resume vs Job Description) × 100

Score Range	Recommendation Tier	Recruiter Advisory Meaning
80.0 – 100.0	strong_match	Candidate meets or exceeds all core skills and experience prerequisites.
65.0 – 79.9	good_match	Candidate satisfies primary criteria with minor qualification gaps.
50.0 – 64.9	moderate_match	Candidate possesses foundational skills but lacks depth or specific stack tools.
0.0 – 49.9	low_match	Significant divergence between candidate background and job requirements.

4. REST API Route Specifications

All endpoints are prefixed under `/api/v1` (except `/health`) and produce standardized JSON responses.

Endpoint: GET `/health`

Description: Public liveness and readiness probe.

Request: No body required.

Response Sample:

```
{"status": "healthy", "service": "smart-hire-ml-service", "version": "1.0.0", "uptime_seconds": 120.4}
```

Endpoint: POST `/api/v1/resume/extract`

Description: Uploads PDF/DOCX resume file (max 10MB) and returns extracted plain text.

Request: multipart/form-data with `file` binary.

Response Sample:

```
{"success": true, "document_type": "pdf", "text": "Jane Doe\nSoftware Engineer...", "page_count": 2}
```

Endpoint: POST `/api/v1/resume/analyze`

Description: Executes full extraction pipeline and returns structured candidate profile.

Request: multipart/form-data with `file` binary.

Response Sample:

```
{"candidate_profile": {"skills": ["python", "docker"], "experience_years": 4.5, "education": ["B.Sc."]}}
```

Endpoint: GET `/api/v1/skills`

Description: Retrieves list of all canonical technical skills supported in controlled dictionary.

Request: No parameters.

Response Sample:

```
{"count": 72, "skills": ["python", "react", "typescript", "docker", "fastapi", "kubernetes", ...]}
```

Endpoint: POST `/api/v1/skills/extract`

Description: Extracts skills from unstructured raw text and normalizes aliases.

Request: `{"text": "Experienced in ReactJS, Node.js, and Amazon AWS"}`

Response Sample:

```
{"skills": ["react", "nodejs", "aws"]}
```

Endpoint: POST `/api/v1/job/analyze`

Description: Normalizes job posting requirements into structured JobProfile for matching.

Request: `{"job_title": "Backend Dev", "description": "...", "required_skills": ["python", "fastapi"]}`

Response Sample:

```
{"job_profile": {"job_title": "Backend Dev", "skills": ["python", "fastapi"], "processed_text": "..."}}
```

Endpoint: POST `/api/v1/screening/evaluate`

Description: Scores a single candidate against job criteria with full transparent explanation.

Request: `{"job": {"title": "...", "required_skills": [...]}, "candidate": {"id": "c1", "skills": [...]}}`

Response Sample:

```
{"match_score": 88.5, "recommendation": "strong_match", "matching_skills": ["python"], "missing_skills": []}
```

Endpoint: POST /api/v1/screening/rank

Description: Evaluates and deterministically ranks a batch of candidates for a job posting.

Request: {"job_id": "j1", "job": {...}, "candidates": [{...}, {...]}

Response Sample:

```
{"job_id": "j1", "total_candidates": 2, "ranked_candidates": [{"rank": 1, "match_score": 91.2, ...}]}
```

5. Module Architecture & Source File Reference

Every file in the ML service has a dedicated single responsibility:

Directory / Package	File Name	Role & Functional Responsibility
app.core	config.py	Pydantic BaseSettings loading .env (ports, CORS, scoring weights).
app.core	logging.py	Configures structured logging and masks PII (emails/phones).
app.core	exceptions.py	Domain error hierarchy mapped to proper HTTP status codes.
app.nlp	normalizer.py	Cleans raw Unicode, strips invalid control chars, standardizes whitespace.
app.nlp	tokenizer.py	Custom tokenizer preserving technical symbols (C++, .NET, Node.js).
app.nlp	section_parser.py	Identifies resume sections (Summary, Experience, Education, Skills).
app.nlp	entity_extractor.py	Extracts degrees, graduation dates, and years of experience.
app.ml	tfidf.py	TfidfModel (unigrams + bigrams) for sparse document representation.
app.ml	similarity.py	BaseSimilarityEngine abstract class & CosineSimilarityEngine.
app.ml	scoring.py	Sub-score formulas for skills, experience, education, and composite score.
app.ml	ranking.py	Deterministic ranking algorithm with stable tie-breaking logic.
app.ml	model_manager.py	Loads and persists serialized models with joblib.
app.services	document_service.py	PyMuPDF (PDF) and python-docx (DOCX) byte-level extraction.
app.services	skill_service.py	Controlled skill matching (70+ skills) & synonym/alias resolution.
app.services	preprocessing_service.py	Coordinates cleaning, section identification, and tokenization.
app.services	candidate_service.py	Orchestrates resume bytes -> structured CandidateProfile.
app.services	job_service.py	Normalizes job posting requirements -> structured JobProfile.
app.services	vectorization_service.py	Maintains active TF-IDF model and caches feature matrices.
app.services	similarity_service.py	Computes cosine & semantic similarity between texts.
app.services	scoring_service.py	Coordinates multi-criteria scoring with configurable weights.
app.services	ranking_service.py	Batch candidate ranking with fault-isolated execution.
app.services	explanation_service.py	Generates human-readable feedback and recommendation tiers.
app.services	gemini_service.py	Optional Google Gemini client for simpler LLM extraction & explanation.
app.data	skills/skills.json	Canonical dictionary of technical skills and categories.
app.data	mappings/skill_aliases.json	Mappings from acronyms/variants to canonical skills.

6. Operational Commands & Environment Setup

The service runs within a local Python virtual environment (.venv). Below are standard operational commands:

Operation	PowerShell Command	Description
Activate venv	.\venv\Scripts\Activate.ps1	Activates project virtual environment with pinned dependencies.
Run Dev Server	python run.py	Starts Uvicorn server with hot-reload at http://localhost:8000.
Direct Run	.\venv\Scripts\python.exe run.py	Launches service without activating terminal environment.
Execute Tests	.\venv\Scripts\python.exe -m pytest	Runs all 89 unit, integration, and end-to-end pytest tests.

Interactive Docs	http://localhost:8000/docs	Opens Swagger UI for live testing of all routes.
------------------	---	--